The Closest Vector Problem as a Mixed-Integer Conic Program

Conic Optimization with `cvxpy` and Clarabel – Practical Assignment

Samir Mansour
BSc in Computer Science, University of Minho

November 1, 2026

Abstract

This report describes the modelling and solving of a bounded version of the Closest Vector Problem (CVP) in a Euclidean lattice. The Euclidean distance is written as a second-order cone constraint, which turns the problem into a mixed-integer second-order cone program (MISOCP). Clarabel does not handle integer variables, so we implemented a best-first branch-and-bound (B&B) in which every node relaxation is a second-order cone program solved by Clarabel through `cvxpy`. The node problem is built once and parametrised (it is DPP), so each node only changes parameter values. The implementation is validated without trusting the solver: an independent solution checker (feasibility, recomputed distance, consistency of the certified bound), exhaustive enumeration on 30 small instances, an independent non-conic exact algorithm (a sphere decoder) and negative tests. At the project sizes we verify that the B&B lower bound and incumbent always *sandwich* the exact optimum. On the required instances ($k \in \{30, 40, 50\}$, $m = 2k$, $\ell \in \{1, 5\}$) the B&B proves optimality for $k = 30$; for $k = 40$ and $k = 50$ it stops at the 120 s limit with a certified gap between 0.20 and 0.93. In all six runs the incumbent it returns equals the exact optimum: the time is spent on *proving* optimality, not on finding the solution. A Mosek code path is included but untested (no licence). Appendix A documents the AI-assisted code review that shaped the final version.

Contents

1 Problem statement **3**

2 Mathematical model **3**
2.1 Why the problem is conic 3
2.2 Choice of solver and consequence 4
2.3 A structural observation about the relaxation 4

3 Implementation **4**
3.1 Instance generation and basis check (Exercise 1) 4
3.2 Continuous relaxation (Exercise 2) 4
3.3 Branch-and-bound with Clarabel (Exercise 3) 5
3.3.1 The DPP node problem 5
3.3.2 The search 5
3.3.3 The convergence trace 6
3.4 Mosek path (Exercise 4, optional) 6

4 Validation (Exercise 5) **7**
4.1 The four layers . . . 7
4.2 The sphere decoder . . . 7
4.3 What `run_tests()` checks . . . 8

5 Experiments (Exercise 6) **8**
5.1 Setup . . . 8
5.2 Results . . . 9
5.3 Convergence . . . 9
5.4 How much does optimising gain over rounding? . . . 10

6 Discussion **10**

A AI-assisted development and code review **11**
A.1 Stage 1: the DPP node problem and the trace . . . 11
A.2 Stage 2: review of our own `ConicBnB` class . . . 12
A.3 Stage 3: validation . . . 12
A.4 Stage 4: experiments and plot . . . 13
A.5 Stage 5: final review of the notebook . . . 13
A.6 Environment issues . . . 13

1 Problem statement

A *Euclidean lattice* in $\mathbb{R}^m$ is generated by a basis of k linearly independent vectors $G = \{g_1, \ldots, g_k\}$, $g_i \in \mathbb{R}^m$:

$$\mathcal{L}_G = \Big\{ x = \sum_{i=1}^{k} z_i \, g_i \;\Big|\; z_i \in \mathbb{Z} \Big\}.$$

The *Closest Vector Problem* asks, for a target $t \in \mathbb{R}^m_+$, for the lattice vector $x \in \mathcal{L}_G$ minimising the Euclidean distance $\mathrm{dist}(x, t) = \|x - t\|_2$. We solve the following version, as required by the assignment:

- the entries $g_{i,j}$ are random reals in $[0, 1]$ (they represent probabilities), and we must check that the g_i really form a basis;
- the integer coefficients are bounded, $0 \le z_i \le \ell$, with $\ell \in \{1, 5\}$;
- the target is $t = G^\top u$ with u drawn uniformly from $[0, \ell]^k$ (real valued, so t is in general *not* a lattice point);
- sizes $k \in \{30, 40, 50\}$ and $m = 2k$.

$G \in \mathbb{R}^{k \times m}$ denotes the matrix whose i-th row is g_i, so a lattice vector is $x = G^\top z$.

Two remarks on the statement.

- **"Basis" means a basis of the lattice, not of $\mathbb{R}^m$.** With $m = 2k$, the k vectors span only a k-dimensional subspace of $\mathbb{R}^m$. Checking that they "form a basis" therefore means checking that they are linearly independent, i.e. $\mathrm{rank}(G) = k$.
- **This is a *bounded* CVP.** The textbook CVP allows any $z \in \mathbb{Z}^k$. The box $0 \le z \le \ell$ turns it into a finite search over $(\ell + 1)^k$ points ($2^{30} \approx 10^9$ or $6^{30} \approx 2 \cdot 10^{23}$ for $k = 30$), which rules out exhaustive search at the project sizes.

2 Mathematical model

2.1 Why the problem is conic

With $x = G^\top z$ the problem is

$$\min_{z \in \mathbb{Z}^k} \|G^\top z - t\|_2 \qquad \text{s.t.} \quad 0 \le z_i \le \ell.$$

The Euclidean norm is not linear, but its epigraph is a second-order cone:

$$\|a\|_2 \le s \iff (s, a) \in \mathcal{Q}^{m+1} = \{(s, a) \in \mathbb{R} \times \mathbb{R}^m \mid \|a\|_2 \le s\}.$$

Introducing a scalar s we obtain the conic model

$$\begin{aligned} \min_{s,\, z} \quad & s \\ \text{s.t.} \quad & \big(s,\; G^\top z - t\big) \in \mathcal{Q}^{m+1}, \\ & 0 \le z_i \le \ell, \quad z_i \in \mathbb{Z}. \end{aligned} \tag{1}$$

Dropping integrality gives a plain second-order cone program (SOCP), solvable in polynomial time by interior-point methods. With integrality, (1) is a mixed-integer SOCP; this is expected to be hard, because CVP is NP-hard.

2.2 Choice of solver and consequence

- **Clarabel** solves continuous conic programs only. We therefore implement the integer search ourselves, with a *branch-and-bound* (B&B) whose node relaxations are SOCPs.
- **Mosek** has a native mixed-integer conic solver, but needs a (student) licence. We include a code path that is active only when Mosek is installed. **It was not run**, so we make no statement about its performance.

2.3 A structural observation about the relaxation

Because $t = G^\top u$ with $u \in [0, \ell]^k$, the point u is feasible for the continuous relaxation and has distance 0. Since G is a basis, $G^\top$ is injective, so $z = u$ is the *unique* minimiser of the relaxation. Hence

> the root relaxation has value 0 and carries no information about the integer optimum;
> the lower bound rises only as branching moves coordinates away from u.

We checked this numerically: for $k = 30$, $\ell = 5$ the Clarabel root solution has value of order 10^{-12} and $\max_i |z_i - u_i| \approx 3 \cdot 10^{-12}$. The test suite also asserts it on a $k = 25$ instance (value $< 10^{-6}$, $\max_i |z_i - u_i| < 10^{-4}$). This explains why B&B is hard on this family: pruning only happens deep in the tree.

3 Implementation

All code is in the notebook `cvp_conic_main.ipynb`, organised in numbered exercises as in the class worksheets. It runs top to bottom (*Restart & Run All*) without errors; the full run takes about 15 minutes with the 120 s time limit. Environment: Python 3.10, `cvxpy` 1.7.3, installed solvers `CLARABEL`, `OSQP`, `SCIPY`, `SCS`. (The OR-Tools import warnings printed at start-up come from an unrelated solver plug-in and do not affect the results.)

3.1 Instance generation and basis check (Exercise 1)

`is_basis(G)` computes the singular values of G and declares the rows independent if the numerical rank (number of singular values above $10^{-10}\,\sigma_{\max}\max(k, m)$) equals k. It also returns the smallest singular value $\sigma_{\min}$, which measures how far the rows are from being dependent. `generate_instance(k, m, ell, seed)`:

- validates its parameters: ℓ and k must be positive integers (Python `bool` is rejected explicitly, since `True` is a subclass of `int`; NumPy integers are accepted), and $m \geq k$; otherwise `ValueError`;
- draws $G \in [0, 1]^{k \times m}$ and redraws it if it is not a basis (practically never); after `max_tries` failures it raises `RuntimeError`;
- draws $u \sim U[0, \ell]^k$ and sets $t = G^\top u$. As $G \geq 0$ and $u \geq 0$, we get $t \in \mathbb{R}^m_+$.

For $k = 30$, $m = 60$, $\ell = 5$, seed 0: rank 30, $\sigma_{\min} = 0.775$. A $\sigma_{\min}$ far from zero means the basis is well away from degenerate, so the numerical rank test is trustworthy.

3.2 Continuous relaxation (Exercise 2)

The relaxation is written with the explicit conic constraint, so the code mirrors (1):

```
z, s = cp.Variable(k), cp.Variable()
prob = cp.Problem(cp.Minimize(s),
                  [cp.SOC(s, G.T @ z - t), z >= lo, z <= hi])
prob.solve(solver=cp.CLARABEL)
```

`solve_relaxation` rebuilds this problem at every call. It is used for Exercise 2 and as the "no DPP" reference in the timing experiment below, but not inside B&B.

3.3 Branch-and-bound with Clarabel (Exercise 3)

3.3.1 The DPP node problem

`ConicBnB` builds the node relaxation *once* in `__init__`. The box bounds are `cvxpy` `Parameter`s:

```
self._z, self._s = cp.Variable(k), cp.Variable()
self._lo, self._hi = cp.Parameter(k), cp.Parameter(k)
self._prob = cp.Problem(cp.Minimize(self._s),
    [cp.SOC(self._s, inst.G.T @ self._z - inst.t),
     self._z >= self._lo, self._z <= self._hi])
assert self._prob.is_dpp(), "node problem must be DPP"
```

The problem is DPP (*disciplined parametrised programming*) because each parameter enters affinely (`z >= lo`) and is never multiplied by a variable; G and t are constants, which is fine since they do not change between nodes. Each node therefore only sets `lo.value`, `hi.value` and calls `solve`, and `cvxpy` reuses the cached canonicalisation instead of rebuilding the conic form. The DPP property is asserted in the code, not only claimed.

`_relax(lo, hi)` returns `None` if Clarabel raises `SolverError` or the status is neither `OPTIMAL` nor `OPTIMAL_INACCURATE`; otherwise it returns $(\max(\text{value}, 0), z^*)$. Solves flagged `OPTIMAL_INACCURATE` are counted in `n_inaccurate`, so they can be reported.

Does DPP pay off? The notebook times the same 40 random boxes with the parametrised problem (after a warm-up call that pays the one-off canonicalisation) and with `solve_relaxation`, which rebuilds the problem every time. Across our runs the parametrised version was about 2× faster per node (measured between 1.9× and 2.4×; about 4 ms against 9 ms per node). The exact factor varies with machine load, so we report it as "about 2×".

3.3.2 The search

- **Node:** integer box $lo \le z \le hi$. Its relaxation value is a valid lower bound for every integer point of the box, since the relaxation contains them.
- **Branching:** take the most fractional coordinate $j = \arg\max_i |z_i^* - \text{round}(z_i^*)|$ and create two children, $z_j \le \lfloor z_j^* \rfloor$ and $z_j \ge \lfloor z_j^* \rfloor + 1$. If the relaxed solution is already integral (fractionality $< 10^{-6}$) the node is not branched; its rounding was already tried when it was created.
- **Eager child evaluation:** both children are solved as soon as they are created, *before* being pushed. The heap therefore holds exact bounds, and a child whose bound is not better than the incumbent is never pushed. This costs a few extra SOCPs compared with pushing children with the parent's bound, but orders the heap better.
- **Incumbent (upper bound):** every relaxed solution is rounded and clipped to its node box (`np.clip(np.rint(z), lo, hi)`), so every candidate is feasible; the best one is the incumbent.
- **Pruning:** a node whose bound is $\ge$ incumbent -10^{-7} is discarded, both when it is created and when it is popped (the incumbent may have improved in between).

- **Node selection:** best-first (smallest lower bound first) with a heap; a counter breaks ties so NumPy arrays are never compared.
- **Global lower bound:** min(incumbent, top of the heap). This is valid for best-first search because a child's relaxation is a restriction of its parent's, so its bound is never smaller.
- **Termination:** an empty heap means the incumbent is optimal. If the time or node limit is reached first, the status is `TIME_LIMIT` or `NODE_LIMIT`, together with the certified lower bound and the gap = incumbent − lower bound. If the limit is hit when all remaining open nodes already satisfy bound ≥ incumbent − tol, there is no real gap and the status is upgraded to `OPTIMAL`.

The result is a `CVPResult` with z, $x = G^\top z$, the distance, the status, the number of SOCPs solved, the time, the certified lower bound, the trace and `n_inaccurate`; the gap is a derived property.

3.3.3 The convergence trace

`solve` fills `trace` with points (time, incumbent, lower bound) through an inner function `record()`. A point is recorded

- at the start (so the plot shows the lower bound starting at 0) and at the end (so the curves meet when optimality is proved);
- whenever the incumbent improves, so no improvement step is lost;
- once every `trace_every` solved SOCPs, using a *threshold*:

```
if improved or nodes >= next_trace:
    record()
    while next_trace <= nodes:
        next_trace += trace_every
```

A test of the form `nodes % trace_every == 0` would be wrong here: two children are solved per iteration, so `nodes` takes only odd values $(1, 3, 5, \dots)$ and would hit only every second multiple of 25 (25, 75, 125, ...). That bug existed in an earlier version and was found during the review described in Appendix A.

Correctness. Each branching step splits an integer box into two disjoint boxes that together contain all its integer points (no integer lies strictly between $\lfloor z_j^* \rfloor$ and $\lfloor z_j^* \rfloor + 1$), and each child box is strictly smaller, so the tree is finite and the search terminates. Pruning only removes boxes whose lower bound is not better than the incumbent, so no better integer point is lost. Hence, when the heap empties, the incumbent is optimal (up to the numerical tolerances).

3.4 Mosek path (Exercise 4, optional)

With Mosek no branch-and-bound is needed: `z` is declared with `integer=True` and $\min \|G^\top z - t\|_2$ s.t. $0 \le z \le \ell$ is passed to the solver (time limit through `MSK_DPAR_MIO_MAX_TIME`). `solve_mosek` returns `None` when Mosek is absent, raises `RuntimeError` if Mosek returns no solution, and otherwise wraps the rounded solution in the same `CVPResult` (status upper-cased, `lower_bound = nan`), so it can go through the same checker. In our environment Mosek was absent (`Mosek available: False`), so this code is **untested**.

4 Validation (Exercise 5)

A solver's own report is not evidence of correctness. Every check below recomputes from the raw instance (G, t, ℓ) and trusts nothing the solver says about itself.

4.1 The four layers

V1. Independent solution checker (`check_solution`). It asserts that:

- z is exactly integral and $0 \le z_i \le \ell$ (feasibility);
- the reported x equals $G^\top z$, recomputed;
- the reported distance equals $\|x - t\|_2$, recomputed;
- the distance is no worse than the naive baseline $z_{\text{base}} = \text{clip}(\text{round}(u), 0, \ell)$ (`baseline_round_u`);
- the certified lower bound is not above the incumbent, $\underline{d} \le d$;
- a result claiming `OPTIMAL` has gap $< 10^{-6}$.

The last two checks were added after the review showed that, without them, a corrupted result with $\underline{d} = d+1$, or a false `OPTIMAL` claim with a gap, was accepted. Both use `np.isnan` so that the Mosek path (`lower_bound = nan`) still passes. Note that the checker certifies *validity*, not optimality; optimality is the job of V2 and V3.

V2. Brute force (`brute_force`): enumerate all $(\ell + 1)^k$ coefficient vectors as the rows of a matrix Z and compute all distances at once as the row norms of $ZG - t$. Exact, but only for small k.

V3. Independent exact algorithm: the sphere decoder (`sphere_decode`). See Section 4.2. It uses no relaxation, no `cvxpy` and no conic solver, so a bug in B&B and a bug in the decoder would have to coincide to agree on a wrong answer.

V4. Negative tests: bad input must be rejected (see below).

4.2 The sphere decoder

Write $B = G^\top \in \mathbb{R}^{m \times k}$ and take a thin QR factorisation $B = QR$, R upper triangular. Since Q has orthonormal columns,

$$\|Bz - t\|_2^2 = \|Rz - y\|_2^2 + \rho^2, \qquad y = Q^\top t, \quad \rho^2 = \|t\|_2^2 - \|y\|_2^2,$$

where ρ is the part of t outside the column space of B ($\rho \approx 0$ here, because $t = G^\top u$). Because R is triangular, the squared distance splits into per-level terms

$$\|Rz - y\|_2^2 = \sum_{i=k}^{1} r_{ii}^2 \, (z_i - c_i)^2, \qquad c_i = \frac{y_i - \sum_{j>i} r_{ij} z_j}{r_{ii}},$$

where the centre c_i depends only on the already fixed $z_{i+1}, \ldots, z_k$. The decoder fixes z_k, then z_{k-1}, …, depth-first (Fincke–Pohst). Every partial sum is a lower bound on the full distance, so a branch is cut as soon as it reaches the best squared distance found so far (the radius). Design choices:

- **Box constraint:** at each level only $z_i \in \{0, \ldots, \ell\}$ is tried.
- **Schnorr–Euchner order:** candidates are tried by increasing $|z_i - c_i|$. The level cost grows with that distance, so once one candidate reaches the radius all the following ones do too, and the loop stops (`break`).

- **Time limit:** checked every 4096 nodes; the decoder reports whether it finished, and every use of it asserts that it did.

The decoder returns the distance computed from the factorisation, $\sqrt{\text{best} + \rho^2}$.

Why not reduce the basis (LLL) first? LLL would make the decoder faster in the unbounded CVP, but its unimodular change of variables $z = Uz'$ turns the box $0 \le z \le \ell$ into a general polytope, so the simple per-level box enumeration would no longer be valid. We therefore enumerate in the original coordinates.

4.3 What `run_tests()` checks

1. **Basis check:** a random 10×20 instance is a basis; a 5×10 set with $g_5 = g_1 + 2g_2$ is rejected; a 12×10 set ($k > m$) is rejected.
2. **Generation properties** (5 seeds, $k = 20$, $\ell = 5$): $G \in [0, 1]$, $t \ge 0$, $t = G^\top u$, $u \in [0, 5]^k$, $m = 40$.
3. **Bad parameters:** $\ell = 0$, $\ell = 1.5$, $\ell =$ `True` and $m = 3 < k$ all raise `ValueError`.
4. **Root relaxation** ($k = 25$): value $< 10^{-6}$ and $z^* \approx u$.
5. **B&B vs brute force** on 30 small instances ($(k, \ell) \in \{(4, 1), (5, 2), (6, 2), (6, 3), (7, 2), (8, 1)\}$, 5 seeds each): B&B returns `OPTIMAL`, passes the checker and matches the exhaustive minimum; the *sphere decoder is also checked against brute force* on the same instances, so the reference is itself validated before it is trusted at large sizes. Largest $|B\&B - \text{brute force}| = 7.8 \cdot 10^{-16}$.
6. **B&B vs sphere decoder** on 5 medium instances ($k = 12$ to 20): same optimum, `OPTIMAL`, checker passes.
7. **Node limit:** with `node_limit=20` the result is `NODE_LIMIT`, still valid, with $\underline{d} \le d \le d_{\text{base}}$.
8. **Sandwich at the project sizes:** for all six configurations, with a 10 s limit, the B&B bounds must enclose the exact optimum d^* from the sphere decoder,

$$\underline{d}_{B\&B} \;\le\; d^* \;\le\; d_{B\&B},$$

   and an `OPTIMAL` claim must match d^*. This is the check that matters most, because $k = 30$–50 is exactly where B&B stops early and where a wrong bound would go unnoticed.

All tests pass.

5 Experiments (Exercise 6)

5.1 Setup

For each $k \in \{30, 40, 50\}$, $m = 2k$, $\ell \in \{1, 5\}$ we generate one random instance (seed 0) and run the conic B&B with a time limit of 120 s. On the same instance we run the sphere decoder (exact) and the baseline round(u). Inside the experiment loop the code asserts that the result passes `check_solution`, that the sphere decoder finished, and that the sandwich $\underline{d} \le d^* \le d$ holds. The results are also saved to `cvp_results.json`. The timings come from a shared machine and vary between runs.

5.2 Results

k	m	ℓ	status	dist	lower b.	gap	nodes	time (s)	exact	t (s)	round(u)
30	60	1	OPTIMAL	2.9304	2.9304	0.0000	3081	12.8	2.9304	0.02	6.8266
30	60	5	OPTIMAL	3.0611	3.0611	0.0000	8935	37.3	3.0611	0.13	9.6306
40	80	1	TIME_LIMIT	3.9763	3.6602	0.3161	18109	120.0	3.9763	0.90	8.0986
40	80	5	TIME_LIMIT	3.7265	3.5229	0.2035	18361	120.0	3.7265	0.39	5.1580
50	100	1	TIME_LIMIT	4.4979	3.9465	0.5514	12593	120.0	4.4979	1.04	26.3432
50	100	5	TIME_LIMIT	4.7071	3.7821	0.9250	13083	120.0	4.7071	4.89	4.7071

Table 1: Conic branch-and-bound (Clarabel, 120 s limit) against the exact sphere decoder. "dist" is the best solution found by B&B; "lower b." is the certified lower bound; "gap" is their difference; "nodes" is the number of SOCPs solved; "exact" and "t" are the optimum and time of the sphere decoder; round(u) is the naive baseline.

The *true gap* $d_{B\&B} - d^*$ is between $-2.5 \cdot 10^{-11}$ and 0 in all six rows: zero up to floating-point noise (the tiny negative values come from computing one distance from G and the other from R).

How to read the table:

- For $k = 30$ the B&B *proves* optimality (12.8 s and 37.3 s), and the value equals the exact reference.
- For $k = 40$ and $k = 50$ the B&B does *not* prove optimality within 120 s. It returns the best point found, with a certified gap between 0.20 and 0.93.
- **In all six runs the incumbent is the exact optimum.** At $k = 40, 50$ the whole time budget is spent on the *proof*: the lower bound starts at 0 and climbs slowly. This is not guaranteed in general: in a shorter test run (15 s) the $k = 40$, $\ell = 5$ instance stopped with incumbent 3.7782 against the optimum 3.7265. The incumbent is only a certified upper bound, which is why we compare it with the reference.
- Fewer nodes are solved at $k = 50$ than at $k = 40$ in the same 120 s, because each SOCP is larger (101 cone dimensions instead of 81).
- The six B&B runs use one instance each (seed 0), so node counts and times are single measurements, not averages.

Numerical note. In one run ($k = 50$, $\ell = 5$) Clarabel reported one node solve as `optimal_inaccurate` (1 of 13 083). We kept its result; an inaccurate relaxation is not strictly a certified bound, so in principle the bound at that node could be slightly off. The sandwich check against the sphere decoder still holds for that run, so the reported lower bound is consistent with the true optimum.

5.3 Convergence

Figure 1 shows the incumbent and the best open lower bound against time, drawn as step functions (the bounds only change at the recorded points) and with the y-axis capped at $1.5\,d^*$ (otherwise the first incumbent, e.g. 26.3 at $k = 50$, $\ell = 1$, would squash the rest). The incumbent reaches the optimum early, whereas the lower bound rises slowly from 0: the gap closes only for $k = 30$. This is the behaviour predicted by the structural observation.

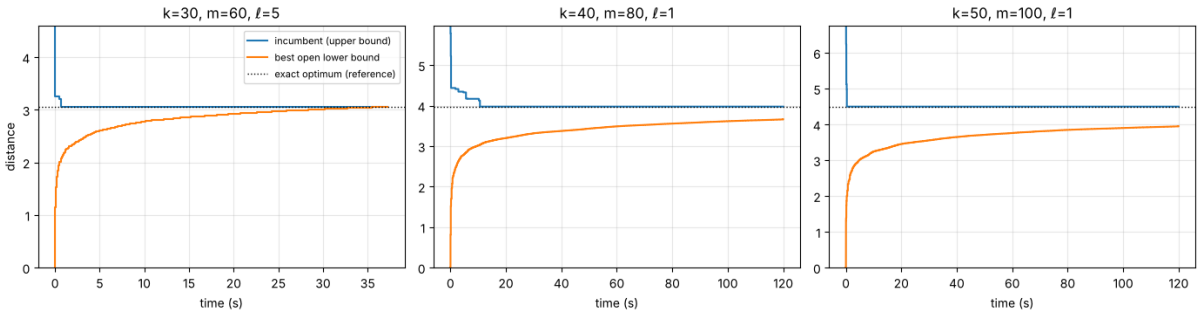


Figure 1: Convergence of the conic B&B for $(k, \ell) = (30, 5)$, $(40, 1)$, $(50, 1)$. Dotted line: exact optimum from the sphere decoder. The traces come from the runs of Table 1.

5.4 How much does optimising gain over rounding?

Over 5 random seeds per configuration (exact optimum from the sphere decoder):

k	ℓ	mean optimum	mean round(u)	ratio
30	1	2.993	5.675	0.53
30	5	3.097	5.508	0.56
40	1	3.926	7.895	0.50
40	5	3.962	8.555	0.46
50	1	4.830	14.627	0.33
50	5	4.772	10.561	0.45

Table 2: Mean optimal distance against the naive baseline of rounding u coordinate-wise.

Rounding u is a poor approximation on average: the optimum is about one half (down to one third) of its distance, so the optimisation is worthwhile. The exception is the single instance $k = 50$, $\ell = 5$, seed 0 in Table 1, where rounding happens to coincide with the optimum (4.7071); this is a property of that instance, not the typical case.

6 Discussion

What the results show.

- The conic B&B is correct: it matches brute force and the independent sphere decoder on all tests, and at the project sizes its bounds always enclose the exact optimum.
- Difficulty grows quickly with k: $k = 30$ is proved optimal in under 40 s, $k = 40$ and 50 are not proved in 120 s. The effect of ℓ is *mixed* in our data: at $k = 40$ the gap is smaller for $\ell = 5$ (0.20) than for $\ell = 1$ (0.32), at $k = 50$ it is larger (0.93 against 0.55). With one instance per configuration we do not claim a trend in ℓ.
- Finding the optimum is easy here; *proving* it is hard. The root bound is 0, so every bit of the lower bound has to be earned by branching.
- Each B&B node costs a full interior-point solve (milliseconds), while a sphere-decoder node costs a few floating-point operations. That is why the specialised enumerator solves all six sizes in under 5 s. This is the usual price of generality: the conic model is short, solver-independent and easy to extend with further constraints, whereas the enumerator is specific to this problem.

Mosek. A native mixed-integer conic solver would replace our B&B entirely. We provide the code but it needs a licence and was not run; we make no performance claim.

Approaches tried but not included. Depth-first node selection used more nodes than best-first on $k = 30$; branching on the lowest or highest index was worse than most-fractional; an LLL basis reduction before branching reduced the node count by roughly a factor of two on instances with $k = 20$ to 30, at the price of turning the box into general linear constraints. None of these changed the outcome for $k = 40, 50$.

Limitations.

- One random instance per configuration in the B&B table (five seeds only for the reference statistics).
- The 120 s limit is arbitrary; with a longer limit the $k = 40$ rows might be proved optimal, but we did not test it.
- One node solve was flagged as inaccurate by Clarabel.
- Mosek was not tested.
- The sphere decoder's distance is computed from the QR factors rather than recomputed from G; recomputing $\|G^\top z - t\|_2$ would make the reference fully independent of the factorisation (the difference is at the 10^{-11} level here).

Possible improvements. Stronger lower bounds deeper in the tree (e.g. better branching rules such as weighting fractionality by $\|g_j\|_2$), a better incumbent heuristic (Babai's nearest-plane rounding instead of coordinate-wise rounding), warm-started node solves, or a native mixed-integer conic solver such as Mosek.

A AI-assisted development and code review

Parts of Exercises 3, 5 and 6 were developed and reviewed in a conversation with the AI assistant Claude (Anthropic) on 31 October 2026. The shared transcript is available at

`https://claude.ai/share/0de3c680-599f-4e27-a6c5-04cfa0b1611b`

We include it because it changed the final code in concrete ways. Below is a summary of each stage, what was found, and what we kept. In every stage the conclusions were checked by running the code, not just by reading it.

A.1 Stage 1: the DPP node problem and the trace

Request. Build Exercise 3 so that the node problem is built once with `cvxpy` parameters for lo, hi (DPP), and add a trace of (time, incumbent, lower bound) for convergence plots; also review the earlier exercises.

Outcome. The assistant first wrote its own reference implementation (a function `branch_and_bound` with a closure-based `build_node_problem`, lazy child evaluation and a `history` list) and tested it against brute force. It confirmed that Exercises 1–2 match the statement and that the analysis is right (root bound exactly 0 because $G^\top$ is injective), and it pointed out:

- `seed`: `int` should be `int | None`; `isinstance(ell, int)` rejected `np.int64`;

- two claims in the markdown had no supporting code at the time (a depth-first comparison and a Mosek code path);
- `solve_relaxation` returns `None` on an infeasible box, so it must not be reused inside B&B;
- suggested report wording: "basis of the lattice, not of $\mathbb{R}^m$", "bounded CVP", report $\sigma_{\min}$.

All three wording suggestions are used in this report.

A.2 Stage 2: review of our own `ConicBnB` class

We then wrote our own class, `ConicBnB`, and asked for a review. The assistant ran it against its reference on $k = 15$ and $k = 30$: same optima, and `is_dpp()` is `True`. The algorithm was sound; the problems were in the trace and in edge cases:

Problem found	Fix in the final code
`nodes % trace_every == 0` fires only on every second multiple, because `nodes` grows by 2 and is always odd (62 trace points instead of ≈ 123 at $k = 30$, $\ell = 1$).	Threshold `next_trace` (Section 3.3.3).
No trace point at the start or end; empty trace (plot crash) for runs under 25 nodes.	`record()` at start and end.
Incumbent improvements between samples merged.	`record()` whenever the incumbent improves.
`TIME_LIMIT` reported even when all open nodes are already prunable.	Status upgraded to `OPTIMAL` in that case.
`n_inaccurate` never reset between `solve()` calls.	Reset at the start of `solve`; also initialised in `__init__`.
No handling of a failed root solve or `cp.SolverError`.	`try/except` in `_relax`; `RuntimeError` if the root fails.
Redundant rounding of an already-integral popped node.	Removed.
`time.time()`, string statuses, unasserted DPP claim.	`perf_counter`, `cp.OPTIMAL`, `assert is_dpp()`.

The review also confirmed two of our design choices as good and worth stating: *eager* child evaluation (exact bounds in the heap, pruning before pushing) and the global lower bound min(incumbent, heap top). On the question "yours or mine?" the recommendation was to keep *our* fixed class: both were equally correct, ours carries a richer result object (`x`, three-way status, `node_limit`, `n_inaccurate`), and it is the code we can defend line by line. The assistant's version was useful only as an independent cross-check.

A.3 Stage 3: validation

We described the four validation layers (checker, brute force, sphere decoder, negative tests). The assistant first produced a long version (a dictionary-of-flags checker, a sphere decoder with a V-BLAST column ordering that roughly halved its node count, 27 negative tests). We found it too long and wrote a compact version (`check_solution` with assertions, `brute_force`, `sphere_decode`, `run_tests`). The assistant ran both and concluded that ours was the better deliverable (about a quarter of the length; 4.5 s instead of 3.5 min), with two real gaps, both shown by deliberately corrupted inputs:

1. **The lower bound was never checked.** A result with $\underline{d} = d + 1$ and a false `OPTIMAL` claim were both accepted. Two assertions were added to `check_solution`.

2. **The sphere decoder was never used at the project sizes**, which is exactly where B&B stops early. The sandwich test $\underline{d} \le d^* \le d$ for all six configurations was added, plus a line validating the sphere decoder itself against brute force, plus $\ell =$ `True` in the bad-parameter loop.

It also explained why LLL cannot be used inside a box-constrained enumeration (Section 4.2).

A.4 Stage 4: experiments and plot

Our experiment and plot cells were run as written. The plot code was correct (the "not plotting" issue came from the 12-minute experiment cell not having finished, or from an interrupted run leaving `traces` incomplete). Changes adopted:

- assert that the sphere decoder finished and that the sandwich holds inside the experiment loop;
- store and print the *true gap* $d_{B\&B} - d^*$; in a 15 s test run the $k = 40$, $\ell = 5$ incumbent was *not* optimal (3.7782 vs 3.7265), so this column is informative;
- report `n_inaccurate`;
- draw the bounds with `drawstyle="steps-post"` and cap the y-axis at $1.5\,d^*$.

A.5 Stage 5: final review of the notebook

Restart & Run All on our uploaded notebook crashed: some stale cells from the assistant's first version still called `dist`, `branch_and_bound`, `build_node_problem` and `res.history`. These cells were removed (Exercises 5 and 6 already cover their purpose), the DPP timing cell and the Mosek cell were rewritten for `ConicBnB`/`CVPResult`, the Exercise 3 markdown was updated to describe `ConicBnB`, and the full notebook was executed with the 120 s limit, producing the numbers in Table 1. The review also flagged statements in the text that the data do not support, which this report handles as follows:

- "difficulty grows with ℓ" is not supported (mixed gaps), so we say the effect of ℓ is mixed;
- "optimising beats rounding" is true on average, but not for $k = 50$, $\ell = 5$, seed 0, which we state explicitly;
- the strongest finding is that the incumbent equals the exact optimum in all six runs, and the time goes into the proof;
- the DPP speed-up varied between runs (1.9× and 2.4×), so we report "about 2×";
- claims about experiments not shown in the notebook (depth-first comparison, other branching rules, LLL) should be kept only if those experiments were actually run.

A.6 Environment issues

Two environment problems came up and were *not* bugs in the code: a `KeyboardInterrupt` during the `cvxpy`/`scipy` import in an early run, and OR-Tools version warnings at start-up (`cvxpy` 1.7.3 expects OR-Tools < 9.15; installing `ortools<9.15` silences them). Neither affects Clarabel or the results.